

«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)

ОТЧЁТ
ПО **УЧЕБНОЙ** ПРАКТИКЕ

Commented [МОЭВМ1]: Для студентов 2го курса –
УЧЕБНАЯ практика
Для студентов 3го курса – ПРОИЗВОДСТВЕННАЯ
практика

Тема: ГЕНЕТИЧЕСКИЕ АЛГОРИТМЫ

Студенты	гр.	В.Н. Волкова, А.Д.
4342		Киселева, Е.С.
		Ковалев
Руководитель		Т.Р. Жангиров

Санкт-Петербург
2026

**ЗАДАНИЕ
НА УЧЕБНУЮ ПРАКТИКУ**

Студенты: В.Н. Волкова, А.Д. Киселева, Е.С. Ковалев

Группа: 4342

Тема практики: Генетические алгоритмы

Задание на практику:

Реализовать генетический алгоритм для поиска всех точек максимума заданного полинома.

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 16.07.2026

Дата защиты отчета: 16.07.2026

Студент	_____	Е.С. Ковалев
Руководитель	_____	Т.Р. Жангиров

АННОТАЦИЯ

Краткое изложение содержания отчёта.

SUMMARY

Аннотация на английском языке.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	5
1 ПОСТАНОВКА ЗАДАЧИ	6
1.1 Предметная область	6
1.2 Задачи практики	6
2 РЕЗУЛЬТАТЫ РАБОТЫ.....	7
2.1. Распределение ролей и структура	7
2.2. Составление списка задач	7
2.3. Создание макета GUI	8
2.4. Третья итерация.....	9
2.4.1 Polynomial.h / Polynomial.cpp.....	9
2.4.2 Types.h	9
2.4.3 GeneticAlgorithm.h / GeneticAlgorithm.cpp	9
2.4.4 GUI.....	11
3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ	13
4 ОТЗЫВ РУКОВОДИТЕЛЯ	14
ЗАКЛЮЧЕНИЕ	15
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	16
ПРИЛОЖЕНИЕ А	17

ВВЕДЕНИЕ

Цель учебной практики — разработать генетический алгоритм, метод поиска оптимальных решений, который основан на принципах естественного отбора. Генетический алгоритм позволяет избежать застревания в локальных экстремумах благодаря работе с популяцией решений и использованию мутаций.

Для этого необходимо решить следующие задачи:

- Написать алгоритм на C++ для поиска всех максимумов заданного полинома.
- Разработать GUI с возможностью задавать параметры алгоритма и видеть пошаговую отрисовку графиков.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Предметная область

Генетические алгоритмы применяются для решения задач оптимизации, комбинаторных задач, задач, в которых нет математического решения или в которых сложная целевая функция, а также для нахождения игровых стратегий, симуляции поведения и машинного обучения.

1.2 Задачи практики

Вариант 6. Задача о поиске максимумов: для заданного полинома $f(x)$ (степень не больше 8). Необходимо с помощью генетического алгоритма найти все точки максимума (локальные и глобальные) на заданном интервале $[l, r]$.

Требование к решению:

- Программа должна иметь GUI.
- Должна быть возможность задать данные через GUI, чтение из файла, случайную генерацию.
- Алгоритмы реализуются самостоятельно без использования готовых решений.
- Настройкой параметров алгоритмов должна производиться пользователем.
- Пошаговая визуализация поиска решения (как меняется аппроксимирующая функция, текущий экстремум, текущее решение оптимизационной задачи в зависимости от популяции). Должен быть выбор переключения между разными текущими решениями.
- Возможность перейти к конечному решению пропустив отображение всех шагов.
- Должен присутствовать график изменения функции качества решения от номера популяции. График дополняется с каждым шагом алгоритма.

2 РЕЗУЛЬТАТЫ РАБОТЫ

В рамках первых итераций были распределены роли внутри бригады, составлен список задач, выбраны языки программирования и инструменты для работы, спроектирована первичная структура проекта и подготовлен макет графического интерфейса пользователя.

2.1. Распределение ролей и структура

Роли в бригаде были распределены следующим образом:

- Волкова В.Н. – разработка логики ГА, связь алгоритма с графическим интерфейсом;
- Киселева А.Д. – разработка GUI и графиков, работа с Qt;
- Ковалев Е.С. – разработка ГА, составление отчета.

Для разделения вычислительной логики и визуализации была спроектирована следующая архитектура проекта:

- core/ — вычислительное ядро генетического алгоритма, независимое от графического интерфейса. Включает модули вычисления функции $f(x)$ (Polynomial.h/cpp), логику селекции, скрещивания, мутации, образования ниш (GeneticAlgorithm.h/cpp), а также структуру снимка популяции для сохранения лучших особей и показателей приспособленности (GenerationSnapshot.h).
- gui/ — слой графического интерфейса пользователя. Взаимодействует только со снимками состояния популяции (snapshot), без прямого доступа к математической логике алгоритма.
- io/ — модуль ввода/вывода, отвечающий за работу с файлами, случайную генерацию и валидацию данных.

2.2. Составление списка задач

На этапе планирования был сформирован перечень задач, разделенный на два основных направления разработки.

Задачи по реализации алгоритма (C++):

1. Определить целевую функцию, структуру индивидуума и его генов, а также критерий остановки алгоритма.
2. Выбрать метод отбора.
3. Реализовать несколько методов скрещивания и мутации (в зависимости от вида хромосомы) для обеспечения возможности выбора.
4. Определить тип и суть применяемых ограничений (жесткие, мягкие) и обосновать их использование.
5. Реализовать механизм образования ниш для нахождения всех максимумов функции.
6. Обеспечить возможность считывания входных данных из файла и сохранение итогового результата в выходной файл.

Задачи по реализации графической части:

1. Определить внешний вид приложения и расположение элементов управления.
2. Разработать динамически меняющиеся графики, отражающие процесс поиска решения алгоритмом в текущий момент времени.

2.3. Создание макета GUI

В ходе выполнения задач графической части был подготовлен первичный макет графического интерфейса пользователя (см. рис. 1).

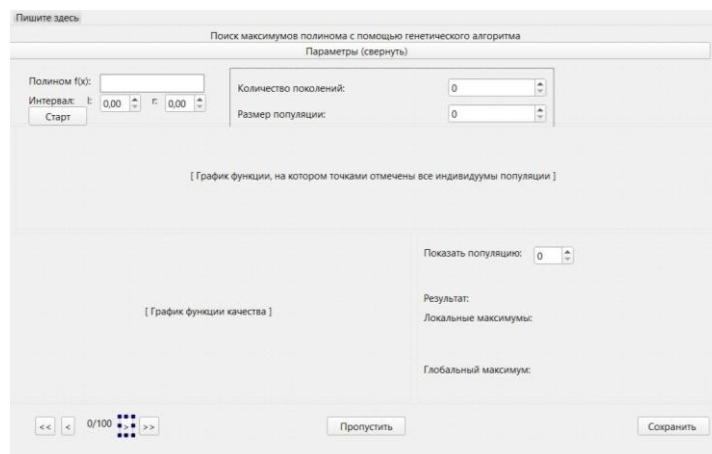


Рисунок 1 – макет GUI

2.4. Третья итерация

В рамках третьей итерации был дописан генетический алгоритм и графический пользовательский интерфейс.

2.4.1 Polynomial.h / Polynomial.cpp

Файлы содержат функцию `evaluatePolynomial()`. Функция принимает на вход вектор коэффициентов `coeffs` и значение точки `x`. Алгоритм вычисляет значения полинома.

2.4.2 Types.h

Определяет структуры данных:

`Individual`: содержит координату `x`, сырое значение приспособленности и скорректированное значение, необходимое для метода ниш.

`ProblemDefinition`: структура для хранения входных данных задачи (вектор коэффициентов полинома, границы диапазона `l` и `r`).

`GAParameters`: параметры алгоритма (размер популяции, вероятности операторов, радиус ниши). Используются перечисления `SelectionType`, `CrossoverType`, `MutationType` для выбора стратегии работы алгоритма.

`GenerationSnapshot`: структура для хранения состояния популяции на конкретной итерации, включающая средние и лучшие показатели, а также список найденных экстремумов.

2.4.3 GeneticAlgorithm.h / GeneticAlgorithm.cpp

Основной класс алгоритма.

`initializePopulation()`: Заполняет популяцию случайными значениями из диапазона `[l, r]` с использованием `std::uniform_real_distribution`. Для каждой особи рассчитывается `rawFitness`.

`step()`: Главный метод итерации. Алгоритм:

- Расчет приспособленности с учетом ниш.
- Сохранение элиты (лучшей особи).
- Селекция родителей.
- Скрещивание.

- Мутации.
- Обновление популяции и сохранение состояния.

`computeSharedFitness()`: Механизм ниш. Приспособленность каждой особи изменяется в зависимости от плотности расположения других особей в заданном радиусе.

Селекция:

- `tournamentSelect`: выбирает 2 случайные особи и возвращает лучшую из них.
- `rouletteSelect`: распределяет вероятность выбора пропорционально значению `fitness`.

Кроссовер (`arithmeticCrossover`, `blxCrossover`):

- `arithmeticCrossover`: создает потомка как среднее арифметическое родителей.
- `blxCrossover`: генерирует потомка случайно из интервала, расширенного за пределы значений родителей, что увеличивает разнообразие популяции.

Мутация:

`gaussianMutate`: добавляет к гену значение из нормального распределения.

`uniformMutate`: заменяет значение гена случайным числом в допустимом диапазоне.

`extractMaxima()`: Метод для извлечения экстремумов.

- Особи сортируются по убыванию `rawFitness`.
- Особь добавляется в список максимумов, если расстояние до ближайшего уже добавленного максимума превышает `nicheRadius`.
- Дополнительно проводится проверка сравнение значения функции в точке x со значениями в $x-0.1$ и $x+0.1$, чтобы исключить точки, находящиеся на склонах функции.

console_main.cpp: функция, позволяющая запускать алгоритм в консоли. С его помощью были проведены тестирование и отладка алгоритма.

2.4.4 GUI

По созданому графическому шаблону реализована визуальная составляющая программы. Все элементы интерфейса расположены внутри главного окна, поделенного на несколько секций (см. рис. 2):

1. Сверху располагается секция ввода данных: ввод полинома и интервала находятся в открытой части секции, при нажатии на кнопку «Параметры» разворачивается дополнительное поле для ввода параметров генетического алгоритма. В разворачиваемом поле присутствует кнопка «Загрузить», позволяющая загрузить данные из файла формата .txt, и кнопка «Сгенерировать», заполняющая параметры случайными данными.

2. Под секцией ввода данных располагается секция, содержащая график полинома с отмеченными локальными максимумами всех популяций.

3. Ниже расположена секция функции качества, отражающая зависимость качества поколения от его порядкового номера.

4. Справа от графика функции качества располагается секция отображения конкретного поколения и результатов работы алгоритма.

5. Ниже располагается секция, содержащая кнопки, позволяющие визуализировать решение пошагово, кнопку «Пропустить», предоставляющую возможность сразу получить готовое решение, и кнопку «Сохранить» для сохранения результатов.

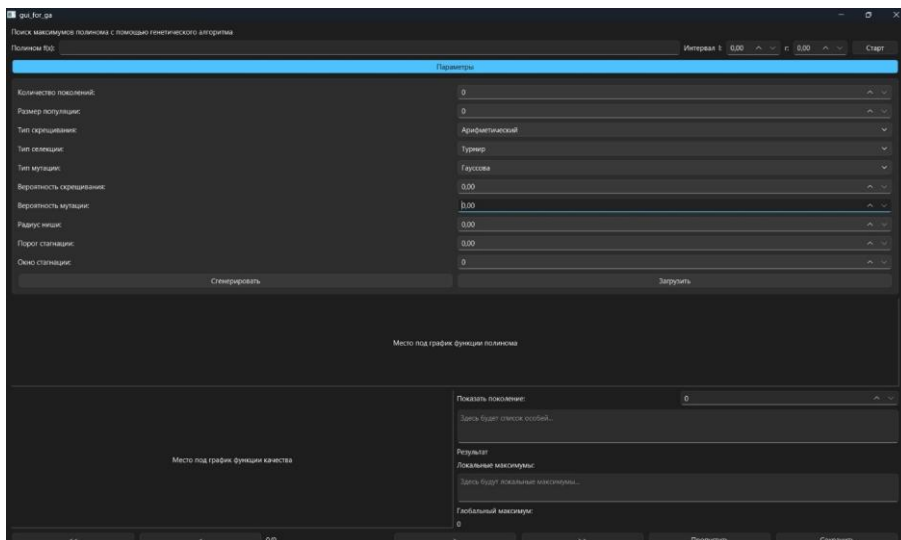


Рисунок 2 — Графический интерфейс

3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

В процессе работы над учебной практикой применялись следующие языки программирования, библиотеки, фреймворки и инструменты:

- Язык программирования C++ — использован для написания самого алгоритма. Выбор обусловлен высокой производительностью языка.
- Фреймворк Qt и среда разработки Qt Creator — применялись для реализации графического интерфейса пользователя.
- Система контроля версий git — использовалась для организации совместной работы внутри бригады.

4 ОТЗЫВ РУКОВОДИТЕЛЯ

Если прохождение практики было по кафедральным темам достаточно добавить формулировку:

По результатам работы учебная практика оценена на <ваша_оценка>.

Если прохождение практики было по своей теме, то необходимо добавить скан отзыва руководителя с оценкой и подписью, скан должен быть в высоком качестве на всю страницу.

Для прохождения автоматической проверки для скана необходима подпись и ссылка в тексте, т.к. он считается рисунком.

Отзыв руководителя с оценкой (см. рис. X).

Отзыв о прохождении учебной практики

<ФИО студента>, студент(ка) группы <номер группы>, второго курса бакалавриата Санкт-Петербургского электротехнического университета "ЛЭТИ" им В.И. Ульянова, проходил(а) учебную практику по теме "<тема практики>" с <дата начала> по <дата окончания>.

В течение практики обучающийся(аяся) <ФИО студента> выполнял <задание на практику>.

В результате практики обучающийся(аяся) <результат практики>. Получаемую работу обучающийся(аяся) <ФИО студента> выполнял <характеристика личных качеств практиканта>.

По результатам работы <ФИО студента> учебную практику оцениваю на <оценка: Отлично, Хорошо, Удовлетворительно, Неудовлетворительно>.

Подпись руководителя практики:

<Должность><дата>

<подпись>/<ФИО>

Рисунок X — Отзыв руководителя

ЗАКЛЮЧЕНИЕ

Заключение содержит краткое описание результатов по каждой задаче, обозначенной в разделе «Постановка задачи», результаты работы в целом, согласно обозначенной цели и пр.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. The State of the Octoverse [Электронный ресурс]. URL: <https://octoverse.github.com> (дата обращения: 28.04.2021).
2. Бобкова, О.В., Давыдов, С.А., Ковалева, И.А., Плагиат как гражданское правонарушение / О.В. Бобкова, С.А. Давыдов, И.А. Ковалева // Патенты и лицензии. – 2016. – № 7. – С. 31-37.

ПРИЛОЖЕНИЕ А